Octobre 2026

Rapport de TP

Detection Engineering & Dashboards (SOC L1)

Pape Aldiouma SARR
L3 Cybersecurite - Module : Ethical Hacking / SOC

Rapport de travaux pratiques : TP 4, Detection Engineering & Dashboards. Parcours SaaS : RBAC, import de regles et dashboards, alerting, tuning, regles maison, EQL et detection-as-code.



Republique du Senegal, Un Peuple, Un But, Une Foi
Ministere de l'Enseignement Superieur, de la Recherche et de l'Innovation
Direction Generale de l'Enseignement Superieur Prive
Institut Superieur de Management
Annee academique 2026-2027

Table des matieres

Partie I : Presentation du travail....3
Partie II : Espace et RBAC....4
Partie III : Import des regles et dashboards....6
Partie IV : Alerting et tuning....8
Partie V : Regles maison (Etape 6)....10
Partie VI : EQL et detection-as-code....15
Partie VII : Defis (autonomie)....17
Conclusion....20

Partie I : Présentation du travail

Contexte

A l'issue du TP2, la plateforme SOC-LAB-GLACIER (Elastic Cloud Hosted) contient des données simulees ingerees et interrogeables (1505 documents repartis sur deux index). Ce TP4 transforme ces capacités de recherche en détection opérationnelle : importer des règles de détection et des dashboards prets a l'emploi, mettre en place le RBAC (rôle et utilisateur analyste a moindre privilege), faire fonctionner l'alerting, tuner une règle bruyante, créer quatre règles maison, exécuter une requête de correlation EQL, et appliquer la logique détection-as-code (export, versioning).

Objectifs pédagogiques

- Importer des Détection Rules et des dashboards dans Elastic Cloud.
- Gérer le RBAC : créer un rôle et un utilisateur analyste.
- Faire fonctionner l'alerting (règles activees -> alertes).
- Tuner une règle trop bruyante (seuil, exception).
- Créer ses propres règles (Custom query, Threshold, EQL).
- Appliquer la logique détection-as-code (exporter / versionner).

Périmètre du rapport

Les sept étapes du TP (RBAC, import des règles, import des dashboards, alerting, tuning, règles maison, EQL/détection-as-code) et les trois defis d'autonomie (faux positif, dashboard maison, matrice MITRE ATT&CK) ont été menes a terme. Ce rapport documente chaque étape, les obstacles réels rencontres (notamment un bug d'import des objets Lens et un defaut de couverture d'index sur les règles maison), ainsi que les corrections apportees.

Partie II : Espace et RBAC

Choix du Space

Conformement a la recommandation de l'enonce ("le plus simple pour éviter tout blocage : rester dans le Space Default"), aucun Space dedie n'a été cree : toute la configuration de ce TP (rôles, utilisateur, règles, dashboards) reside dans le Space Default, qui reutilise directement la Data View soc-lab-events créée au TP2.

Création du rôle soc_analyst_readonly

Un rôle au moindre privilege a été cree via Stack Management > Security > Rôles. La première tentative a introduit une erreur de compréhension de l'interface : les features Kibana (Discover, Dashboard, Security) avaient été saisies comme un second bloc de privileges Elasticsearch (index privileges) au lieu de la section distincte "Couche applicative" (Kibana privileges). L'erreur a été corrigée en supprimant ce bloc et en utilisant le bouton "Affecter a l'espace", qui a permis de configurer un contrôle granulaire par sous-fonctionnalité (le plan trial de ce cluster n'offre pas de bascule globale par groupe : le toggle principal de la catégorie Security ne descend pas automatiquement sur ses sous-fonctionnalités comme Alerts, Cases ou Timeline, chacune devant être réglée individuellement).

Couche	Ressource	Droit accorde
Elasticsearch (index)	soc-lab-events-*, logs-*	read, view_index_metadata
Kibana (feature)	Discover	Read
Kibana (feature)	Dashboard	Read
Kibana (feature)	Security > Alerts	Read
Kibana (feature)	Toutes les autres fonctionnalités	None

Création de l'utilisateur soc-analyst

Un utilisateur soc-analyst a été cree (Stack Management > Users) et associe au rôle soc_analyst_readonly. La création et la configuration ont été vérifiées a l'écran a chaque étape avant validation finale.

Réponses aux questions 3

Q3.1 - Pourquoi un utilisateur dedie plutôt que elastic ? Le compte elastic est un superutilisateur avec des privileges illimites sur le cluster entier (configuration, suppression d'index, gestion des utilisateurs...). L'utiliser au quotidien pour du triage d'alertes viole le principe de moindre privilege, rend impossible toute traçabilité individuelle des actions (tous les logs porteraient le même compte) et expose un risque disproportionne si les identifiants fuitent.

Q3.2 - Moindre privilege applique a un analyste L1 ? Un analyste L1 doit pouvoir consulter les événements (Discover), visualiser les dashboards et lire les alertes de sécurité -- rien de plus. Il n'a pas besoin de modifier les règles de détection, de gérer les utilisateurs ou d'administrer le cluster. Le rôle soc_analyst_readonly matérialise exactement ce périmètre : lecture seule sur les index de données, et accès Kibana limite a Discover / Dashboard / Alerts en lecture.

Point de contrôle 1

Un utilisateur soc-analyst existe, associe au rôle soc_analyst_readonly dont les droits (lecture seule sur les index SOC-LAB et sur les fonctionnalités Discover, Dashboard et Alertes) ont été verifies a l'écran.

Partie III : Import des règles et des dashboards

Import des 15 règles de détection

Les règles ont été importées via Sécurité > Règles > Importer des règles, en choisissant deliberement le fichier soc-lab-détection-rules-cloud.ndjson plutôt que la version standard soc-lab-détection-rules.ndjson. Les deux fichiers ont été inspectes prealablement : leur seule différence structurelle est le champ from de chaque règle (now-6m / now-11m / now-16m dans la version standard, uniformement now-90d dans la version cloud). Cette différence est determinante : les données du TP2 ont été uploadees dans le passe (datees du jour de l'upload), non générées en flux temps réel. Une règle fenetree sur now-6m ne reinterrogerait donc jamais ces événements et ne declencherait aucune alerte. La version cloud, avec une fenêtre de récupération elargie a 90 jours, couvre le dataset uploade.

Les 15 règles prefixees SOC-LAB ont été vérifiées comme presentes et actives après import (couvrant Threshold, EQL, Query et une diversite de severites de medium a critical).

Réponses aux questions 4

Q4.1 - Pourquoi deux règles de brute force (Threshold et EQL) ? La règle Threshold (#1, SSH Brute Force) compte un volume d'echecs d'authentification par IP source sur une fenêtre donnée : elle detecte l'intensite de l'attaque (combien de tentatives), independamment de son issue. La règle EQL (#2, Brute Force suivi d'un succès) cherche une séquence temporelle précise -- un échec suivi d'un succès pour la même IP dans les 15 minutes -- et detecte donc specifiquement une compromission réussie, un signal de bien plus haute priorite que le seul volume de tentatives.

Q4.2 - Quelle règle genere le plus de bruit ? D'après l'analyse empirique des 124 alertes générées (Partie IV), la règle SSH Brute Force (Threshold, seuil initial de 10 sur 5 minutes) était la plus bruyante avant tuning : son seuil relativement bas et le compte de service deploy generant un volume d'echecs legitimes expliquaient une part disproportionnee des alertes, ce qui justifie précisément le tuning effectue en Partie IV.

Point de contrôle 2

15 règles prefixees SOC-LAB sont importées et visibles dans Sécurité > Règles.

Import des 6 dashboards -- obstacle rencontre

L'import des dashboards (Stack Management > Saved Objects > Import, fichier soc-lab-dashboards.ndjson, 48 objets dont 42 visualisations Lens et 6 dashboards) a echoue de manière constante avec une erreur generique "Internal Server Error". Un diagnostic methodique par isolation a été mene pour en identifier la cause :

- Import du fichier complet (48 objets) : échec.
- Import des 6 dashboards seuls (sans les 42 objets Lens) : succès.
- Import des 42 objets Lens seuls : échec identique.
- Import d'un seul objet Lens extrait : échec identique.
- Même objet avec le champ coreMigrationVersion retire : échec identique.
- Objet Lens minimal écrit a la main : échec identique.

- Export puis reimport d'un objet existant quelconque (test de contrôle) : succès, confirmant que le mécanisme d'import lui-même fonctionne.

Cette isolation converge vers une conclusion : le type d'objet sauvegarde lens est specifiquement casse a l'import sur ce cluster, independamment de son contenu -- un problème vraisemblablement lie a une incompatibilite de version (les objets exportes portent un coreMigrationVersion 8.15.3, tandis que le cluster tourne en version 9.5.5) ou a l'incident "Partial system outage" signale sur la console Elastic Cloud au debut de la session. Stratégie retenue : les 6 dashboards (sans panneaux Lens) ont été importés avec succès, et la visualisation nécessaire au Defi B a été reconstruite manuellement via l'editeur Lens plutôt que par import (voir Partie VII).

Réponses aux questions 5

Q5.1 - Que se passe-t-il a l'import si la Data View n'existe pas ? Les dashboards s'importent sans erreur bloquante mais referencent un identifiant de Data View introuvable : leurs panneaux restent vides (aucune donnée ne peut être résolue). C'est pourquoi la Data View soc-lab-events (ID personnalise, cree au TP2) est un prerequis explicite avant l'import.

Q5.2 - Comment exporter a nouveau un dashboard modifie ? Stack Management > Saved Objects, cocher le ou les dashboards modifies, puis utiliser l'action "Export" -- ce qui produit un fichier NDJSON pouvant être reimporte ailleurs ou verse dans un dépôt Git, exactement selon la même logique que l'export des règles de détection vu en Partie VI.

Point de contrôle 3

Les 6 dashboards sont importés. Limitation documentée : leurs panneaux Lens sont vides du fait du bug d'import identifie ci-dessus ; un dashboard équivalent fonctionnel a été reconstruit manuellement (Defi B, Partie VII).

Partie IV : Alerting et tuning

Activation des règles et première alerte

Les 15 règles importées ont été activees. Une règle d'injection SQL (SOC-LAB - Tentative d'injection SQL) affichait un statut "Failed". Le diagnostic via l'onglet Résultats d'exécution a revele une erreur de syntaxe KQL précise : Expected "", AND, OR but "(" found, provoquee par des parentheses litterales non echappees dans la requête (url.full:*sleep(*). En KQL, la parenthese est un caractere reserve de groupement logique ; une occurrence litterale dans un motif wildcard doit être echappee par un backslash. La requête corrigée (url.full:*sleep\(* or url.full:*benchmark\(*) a été enregistrée et la règle est repassee au statut "succeeded".

Après un cycle d'exécution (intervalle de 5 minutes sur la fenêtre elargie now-90d), 124 alertes ont été générées et consultees dans Sécurité > Alertes.

Réponses aux questions 6

Q6.1 - Quelles règles se sont declenchees, sur quels hotes/IP ? La répartition empirique des 124 alertes a été extraite par règle et par host.name / source.ip, confirmant que la règle SSH Brute Force (Threshold) était la plus bruyante, suivie des règles liées au compte de service deploy et a l'IP 198.51.100.88 déjà identifiée comme source suspecte au TP2.

Q6.2 - Différence entre une règle et une alerte ? Une règle est une definition statique (requête, planification, seuil, severite) qui s'execute periodiquement. Une alerte est le résultat concret d'une exécution de règle qui a trouve une correspondance : c'est un document a part entière, horodate, associe a des champs d'enrichissement spécifiques, qu'un analyste investigue et fait evoluer dans un workflow de triage.

Q6.3 - Quels champs Kibana ajoute-t-il a une alerte ? Un prefixe kibana.alert.* enrichit chaque alerte : kibana.alert.rule.name et kibana.alert.rule.uuid (identite de la règle a l'origine), kibana.alert.severity et kibana.alert.risk_score (criticite), kibana.alert.status et kibana.alert.workflow_status (état de traitement, ouvert/ferme/acquitte).

Point de contrôle 4

Au moins une alerte est visible (124 au total) ; les règles desactivees après observation ont été vérifiées via le filtre "Règles desactivees" (affichage 1-15 sur 15), confirmant que les horodatages de dernière exécution visibles étaient des données historiques et non de nouvelles executions.

Étape 5 : tuning de la règle SSH Brute Force

La règle SOC-LAB - SSH Brute Force (multiples echecs par IP source) était identifiée comme trop bruyante, generant des alertes pour des echecs legitimes du compte de service deploy. Deux ajustements ont été appliques : le seuil a été élève de 10 a 20 echecs sur 5 minutes, et une exception a été ajoutée sur le champ user.name egal a deploy. Une première tentative d'edition s'est portée par erreur sur la règle EQL de brute force (SOC-LAB - Brute Force suivi d'une connexion réussie), dont le nom commence par les mêmes mots -- l'erreur a été identifiée via la banniere "cette règle contient une instruction de séquence EQL" et corrigée en revenant a la bonne règle Threshold avant de finaliser le tuning.

Réponses aux questions 7

Q7.1 - Reduire les faux positifs ou augmenter la couverture ? Les deux ne sont pas mutuellement exclusifs, mais dans ce cas précis la priorite va a la reduction des faux positifs : une règle qui alerte en continu sur une activité legitime et identifiée (le compte de service deploy) epuise l'attention de l'analyste (fatigue d'alerte) et masque les vrais signaux. Augmenter la couverture n'a de valeur que si les alertes générées restent exploitables ; tuner d'abord le bruit existant est le prealable a toute extension de périmètre.

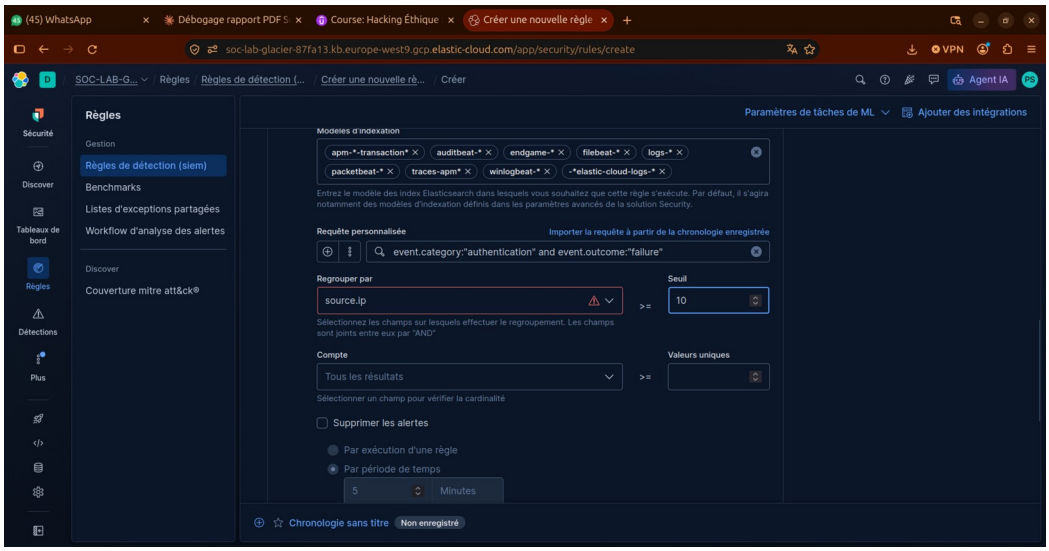
Q7.2 - Une exception est-elle une triche ou une pratique professionnelle ? C'est une pratique professionnelle standard, a condition d'être documentée et justifiee : l'exception ne desactive pas la détection, elle l'affine en excluant une condition précise et connue (ici, un compte de service dont l'activité a été tracee et validée comme legitime), tout en continuant a détecter la même menace pour n'importe quel autre compte. Une exception non justifiee ou trop large serait en revanche un contournement dangereux.

Partie V : Règles maison (Étape 6)

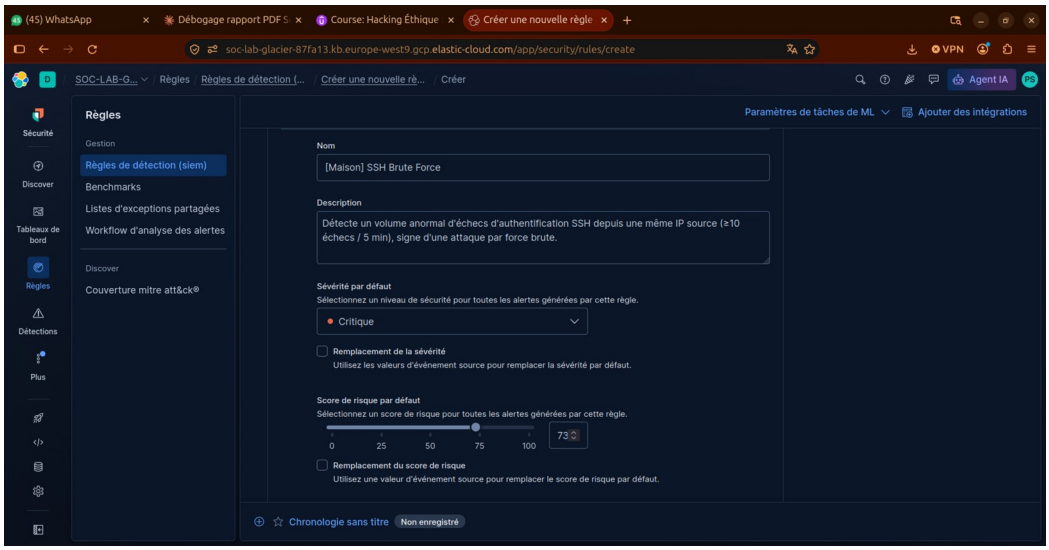
Quatre règles maison ont été créées via Sécurité > Règles > Créer une règle > Création manuelle de règles, toutes avec le prefixe [Maison]. Un obstacle commun a été rencontre et corrige sur l'ensemble des règles : les modèles d'indexation par defaut du formulaire de création (apm-*, auditbeat-*, filebeat-*, logs-*, etc.) ne couvrent pas l'index réel des données simulees (soc-lab-events-*), cree lors de l'upload des datasets au TP2. Sans cet ajout, les règles s'executent sans erreur mais ne trouvent jamais aucun événement. Le modèle d'indexation soc-lab-events-* a donc été explicitement ajoute a chacune des quatre règles.

Règle 1 : [Maison] SSH Brute Force (Threshold)

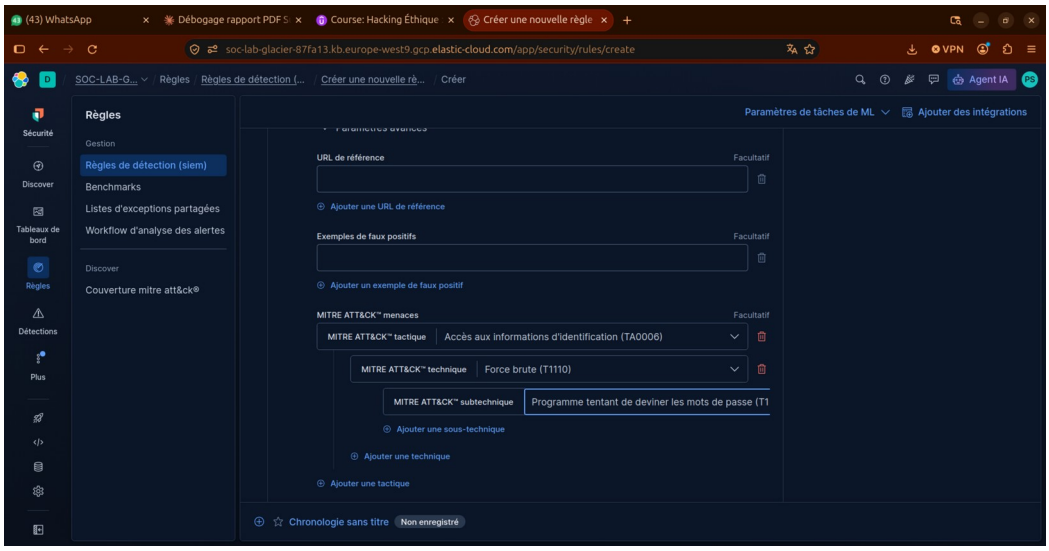
Requête : event.category:"authentication" and event.outcome:"failure", regroupement par source.ip, seuil supérieur ou egal a 10 sur une fenêtre de 5 minutes. Severite High, score de risque 73. Mapping MITRE ATT&CK : tactique Accès aux informations d'identification (TA0006), technique Force brute (T1110), sous-technique Programme tentant de deviner les mots de passe (T1110.001).



Formulaire de la règle Threshold : requête, regroupement par source.ip, seuil >= 10 sur 5 minutes.



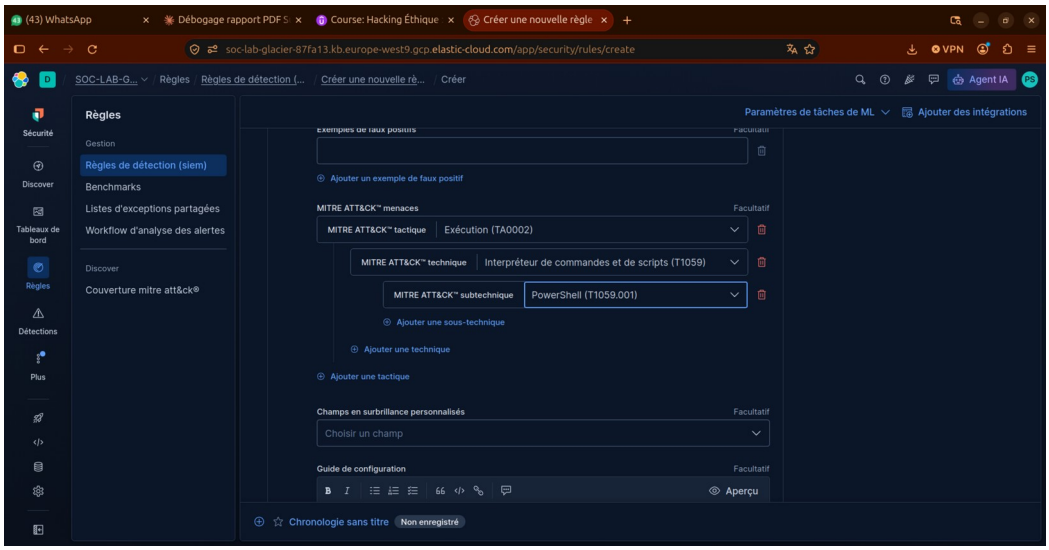
Selection de la tactique MITRE ATT&CK : Accès aux informations d'identification (TA0006).



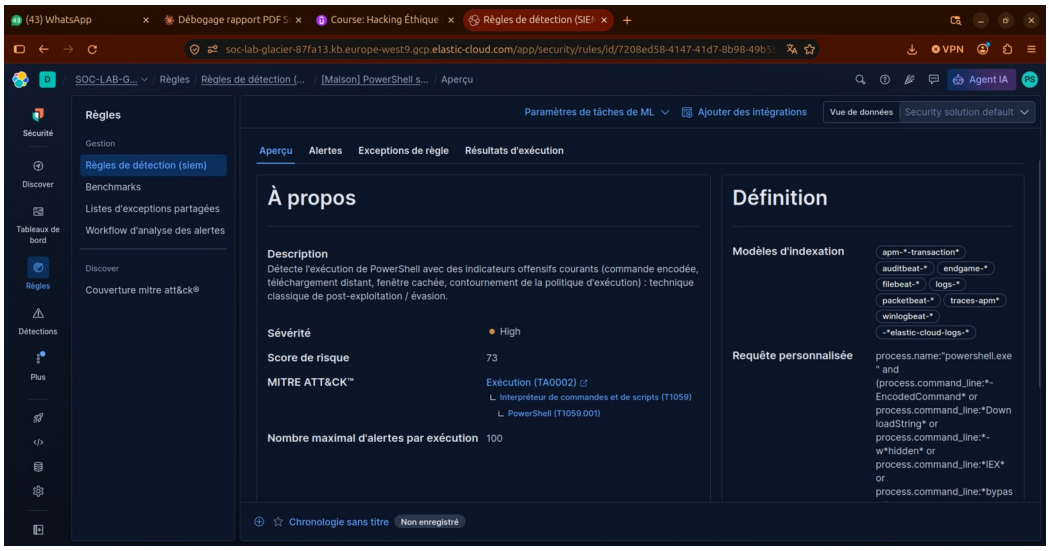
Apercu final de la règle [Maison] SSH Brute Force : severite High, score 73, mapping MITRE complet, planification toutes les 5 minutes.

Règle 2 : [Maison] PowerShell suspect (Custom query)

Requête : process.name:"powershell.exe" and (process.command_line:*-EncodedCommand* or process.command_line:*DownloadString* or process.command_line:*-w*hidden* or process.command_line:*IEX* or process.command_line:*bypass*). Ces motifs sont des indicateurs classiques d'exécution PowerShell offensive : commande encodee en base64, telechargement et exécution de code distant, fenêtre masquee, et contournement de la politique d'exécution de scripts. Severite High, score 73. Mapping MITRE ATT&CK : tactique Exécution (TA0002), technique Interpreteur de commandes et de scripts (T1059), sous-technique PowerShell (T1059.001).



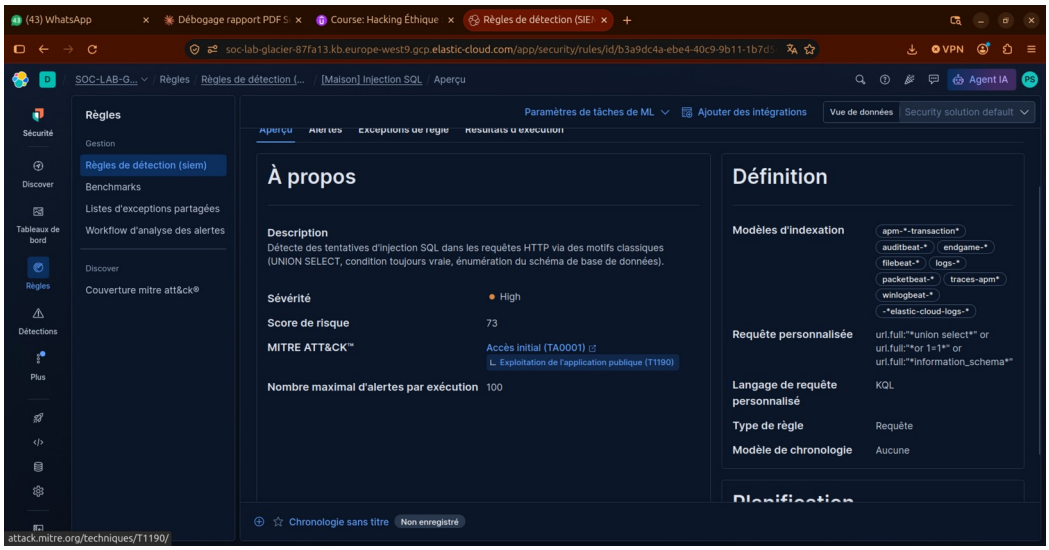
Apercu de la règle [Maison] PowerShell suspect : requête KQL ciblant les indicateurs offensifs PowerShell, severite High, score 73.



Mapping MITRE de la règle PowerShell suspect : Exécution (TA0002) > Interpreteur de commandes et de scripts (T1059) > PowerShell (T1059.001).

Règle 3 : [Maison] Injection SQL (Custom query)

Requête : url.full:"*union select*" or url.full:"*or 1=1*" or url.full:"*information_schema*". Une première écriture avec des espaces echappes par backslash (url.full:*or\ 1=1*) a provoque une erreur "Le KQL n'est pas valide", le caractere = cassant le parseur hors guillemets -- la même classe de problème que celle rencontrée et corrigée a l'étape 4 sur la règle importee équivalente. La correction retenue encadre chaque motif de guillemets plutôt que d'echapper chaque espace individuellement, ce qui traite l'integralite du contenu comme une chaîne litterale a comparer en wildcard. Severite High, score 73. Mapping MITRE ATT&CK : tactique Accès initial (TA0001), technique Exploitation de l'application publique (T1190).

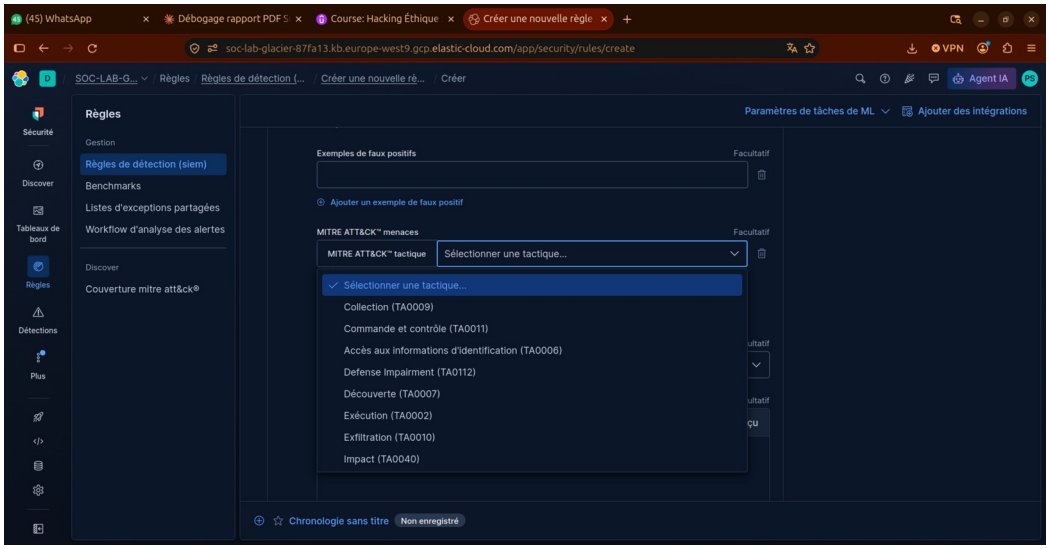


Apercu de la règle [Maison] Injection SQL : requête corrigée entre guillemets, severite High, mapping Accès initial (TA0001) > Exploitation de l'application publique (T1190).

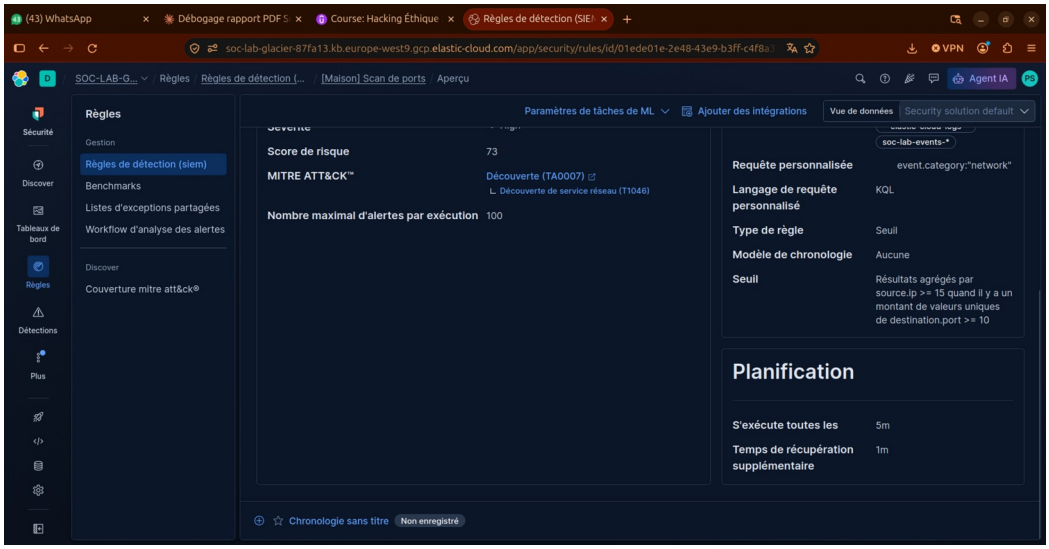
Règle 4 : [Maison] Scan de ports (Threshold + cardinalite)

Requête : event.category:"network", regroupement par source.ip, seuil supérieur ou egal a 15 sur 5 minutes, avec un champ de cardinalite sur destination.port (valeurs uniques superieures ou egales a 10). La cardinalite distingue un scan de ports d'un simple pic de trafic : elle ne compte pas le volume d'événements mais le nombre de ports distincts contactes par une même IP source, signature caractéristique d'une reconnaissance réseau.

Un second obstacle a été rencontre sur cette règle specifiquement : le champ destination.port, pourtant bien présent dans les données (verifie via Discover, ou il apparaît dans la liste des champs disponibles de l'index soc-lab-events-2026.10.19), n'apparaissait pas dans le selecteur de champ de cardinalite tant que le modèle d'indexation soc-lab-events-* n'avait pas été ajoute -- confirmation directe que l'obstacle d'indexation décrit plus haut touchait bien la totalite des règles maison, pas seulement celle-ci. Severite High, score 73. Mapping MITRE ATT&CK : tactique Découverte (TA0007), technique Découverte de service réseau (T1046).



Selection de la technique MITRE pour une règle maison : menu deroulant des techniques rattachees a une tactique.



Details de la règle [Maison] Scan de ports : seuil source.ip >= 15, cardinalite destination.port >= 10, planification 5 minutes.

Réponses aux questions 8

Q8.1 - Pourquoi un risk score en plus de la severity ? La severity (low/medium/high/critical) est une classification discrete en quatre niveaux, pratique pour filtrer et trier visuellement. Le risk score (0-100) est une mesure continue qui permet un classement plus fin entre alertes de même severite et peut être ajuste dynamiquement par des remplacements bases sur les valeurs de l'événement source (par exemple un score plus élève si l'hote cible est un serveur critique), ce que la seule severity ne permet pas d'exprimer.

Q8.2 - Quelle métadonnée est indispensable pour un rapport MITRE ATT&CK ? Le mapping tactique + technique (et sous-technique quand elle existe) rattache a

chaque règle est la métadonnée indispensable : c'est elle qui alimente la matrice de couverture (Partie VII, Defi C) et permet de répondre objectivement a la question "quelles techniques de l'adversaire mon SOC est-il capable de détecter ?".

Point de contrôle 5

Les 4 règles maison sont créées et activees, chacune avec un mapping MITRE ATT&CK complet et une severite/score de risque coherents.

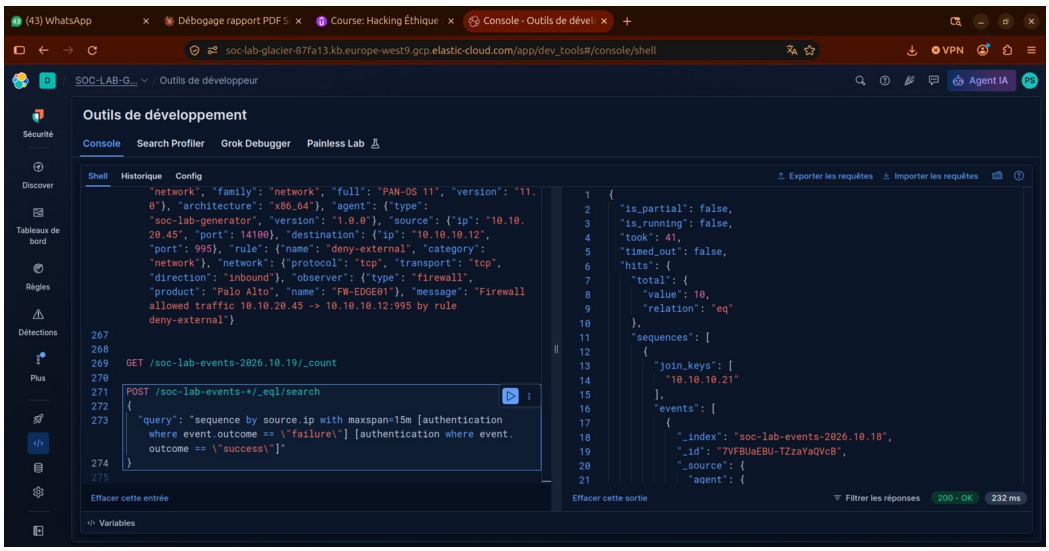
Partie VI : EQL et détection-as-code

Requête de correlation EQL

La requête suivante a été exécutée dans Dev Tools :

```
POST /soc-lab-events-*/_eql/search
{
  "query": "séquence by source.ip with maxspan=15m
    [authentication where event.outcome == \"failure\"]
    [authentication where event.outcome == \"success\"]"
}
```

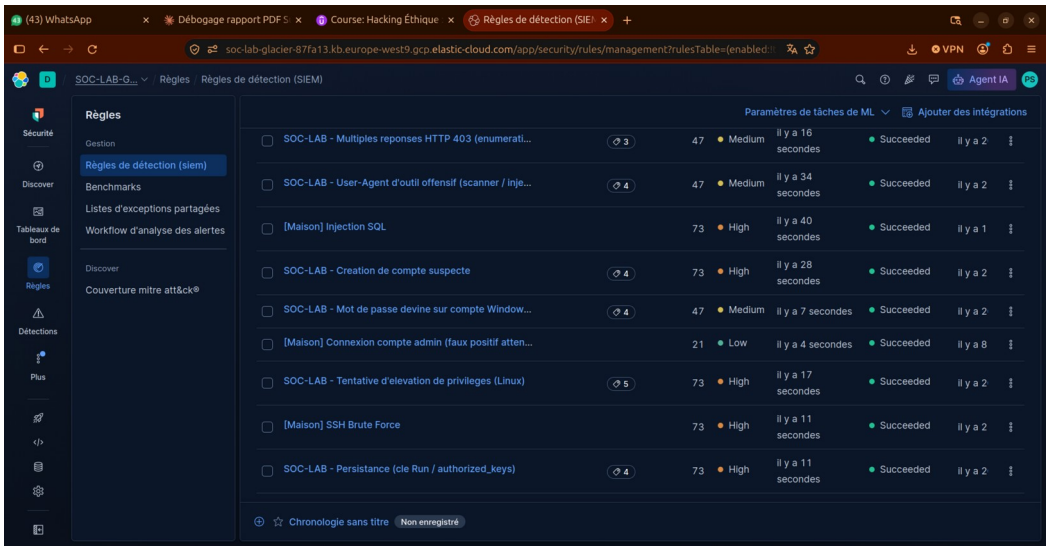
Le résultat a renvoye 10 séquences correspondantes (total.value: 10, relation: eq), chacune identifiant une IP source ayant enchaîne un échec d'authentification puis un succès dans la fenêtre de 15 minutes -- exactement le scénario couvert par la règle importee EQL (#2, Brute Force suivi d'une connexion réussie). Ces 10 séquences constituent des candidats prioritaires d'investigation : un échec suivi d'un succès peut être un faux positif benin (l'utilisateur s'est trompe puis a retape son mot de passe) ou un vrai positif critique (un attaquant ayant fini par deviner ou casser le mot de passe).



Dev Tools : requête EQL de séquence (échec puis succès d'authentification, même source.ip, maxspan 15 minutes) -- 10 séquences trouvées.

Export des règles (détection-as-code)

Les 19 règles (15 importées + 4 maison) ont été exportees au format NDJSON via Sécurité > Règles > Actions groupees > Exporter. Comparee au fichier source soc-lab-détection-rules-cloud.ndjson, cet export differe sur plusieurs points qui materialisent concretement le travail effectue durant le TP : les 4 règles maison portent un rule_id genere par Kibana (UUID) absent du fichier original, et les règles modifiées en cours de TP (Injection SQL avec sa requête corrigée, SSH Brute Force avec son seuil releve a 20 et son exception) présentent un revision supérieur a 0 ainsi qu'un contenu divergent -- la preuve tangible du tuning effectue.



Liste des 19 règles installees, toutes au statut Succeeded après correction des modèles d'indexation.

Réponses aux questions 9

Q9.1 - Qu'apporte l'EQL face au KQL pour la correlation ? KQL recherche des événements independamment les uns des autres : il ne sait pas relier deux événements entre eux dans le temps. EQL permet de definir une séquence (séquence by ... with maxspan=...) : plusieurs conditions devant survenir dans un ordre précis, dans une fenêtre de temps donnée, pour une même entite. C'est indispensable pour détecter des scénarios d'attaque multi-étapes (ici, un échec suivi d'un succès) qu'une simple requête KQL ne peut pas exprimer.

Q9.2 - Comment revue de code et CI s'appliquent-elles aux règles de détection ? Une règle de détection est du code structure (JSON/NDJSON) : elle peut donc suivre les mêmes pratiques qu'un logiciel -- versionnement Git, modification via pull request relue par un pair (pour éviter qu'une règle mal réglée ne genere trop de faux positifs ou, a l'inverse, ne rate des vrais positifs), et tests automatises en intégration continue (validation de la syntaxe KQL/EQL, exécution contre un jeu de données de test pour vérifier la détection attendue avant tout déploiement en production). C'est l'essence du détection-as-code.

Versionner les règles dans Git apporte en particulier : traçabilité (qui a modifie quoi, quand, pourquoi), possibilite de rollback si une règle modifiée degrade la détection, revue par les pairs avant déploiement, tests automatisables en CI/CD, et déploiement reproductible et identique sur plusieurs environnements (dev/test/prod).

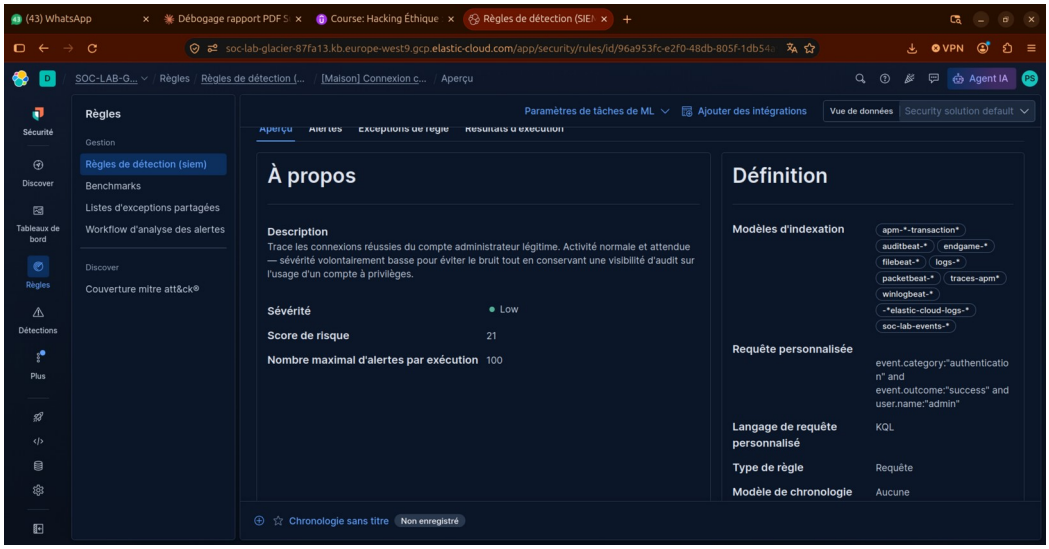
Point de contrôle

Une requête EQL de correlation a été exécutée avec succès (10 séquences trouvées) et les 19 règles ont été exportees en NDJSON, demontrant la compréhension pratique de la logique détection-as-code.

Partie VII : Defis (autonomie)

Defi A : faux positif documente

La règle [Maison] Connexion compte admin (faux positif attendu) a été créée avec la requête event.category:"authentication" and event.outcome:"success" and user.name:"admin", tracant toute connexion réussie du compte administrateur. Contrairement aux quatre règles maison precedentes, celle-ci a volontairement été configurée en severite Low (score de risque 21) : il ne s'agit pas d'une menace mais d'une activité legitime et attendue, qu'un SOC souhaite neanmoins tracer a des fins d'audit sans générer de bruit pour les analystes. Un exemple de faux positif a été documente dans le champ dedie de la règle ("connexion normale de l'administrateur système dans le cadre de ses tâches habituelles").



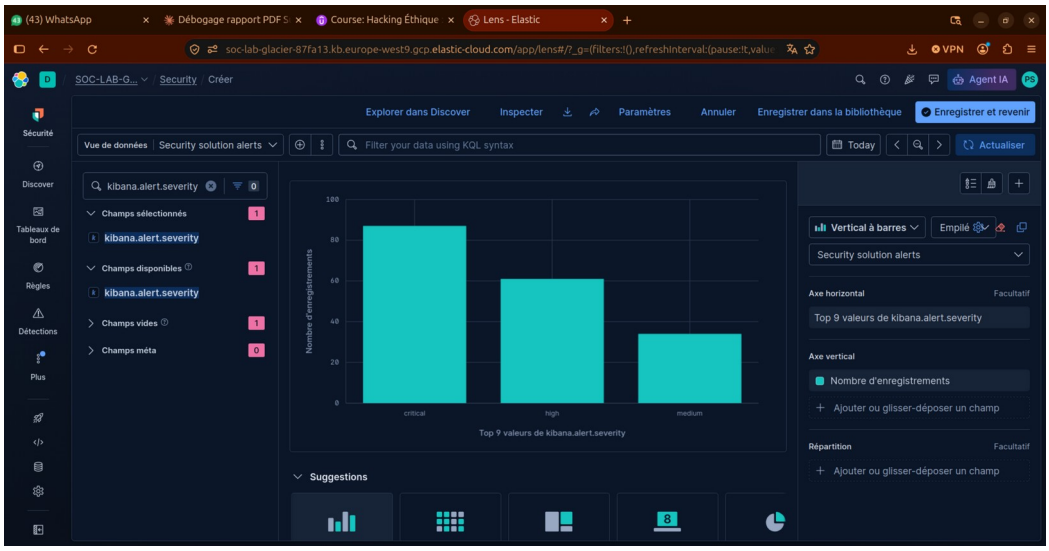
Règle [Maison] Connexion compte admin : severite volontairement Low (score 21), requête ciblant les connexions reussies du compte admin.

Defi B : dashboard maison

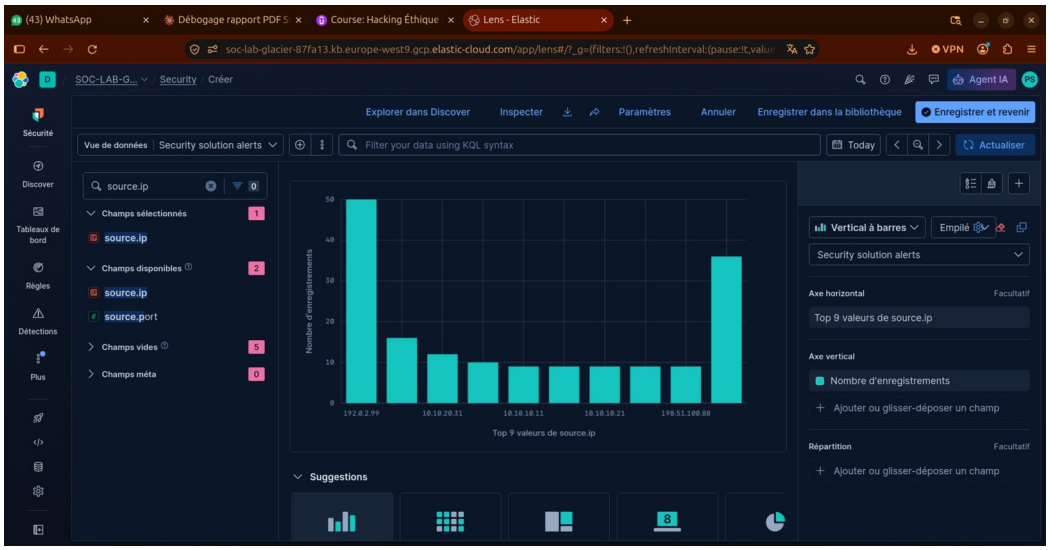
Face au bug d'import des objets Lens documente en Partie III, un dashboard équivalent a été construit manuellement : [Maison] SOC Overview - Alertes, compose de deux visualisations Lens, toutes deux batties sur la Data View Security solution alerts.

Alertes par severite -- répartition par kibana.alert.severity (nombre d'enregistrements), produisant automatiquement un graphique en barres : critical (~87), high (~61), medium (~34).

Top source.ip -- répartition des alertes par source.ip (top 9), faisant ressortir nettement une adresse (192.0.2.99, ~50 alertes) loin devant les suivantes (198.51.100.88, ~36), candidate prioritaire a l'investigation.



Visualisation Lens Alertes par severite : critical ~87, high ~61, medium ~34.



Visualisation Lens Top source.ip : 192.0.2.99 nettement en tête (~50 alertes), suivie de 198.51.100.88 (~36).



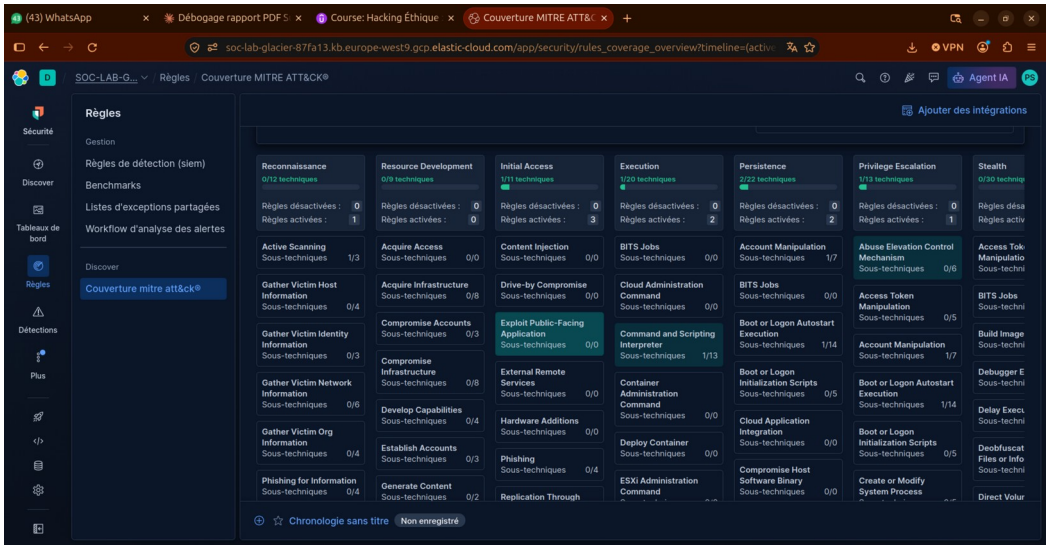
Dashboard final [Maison] SOC Overview - Alertes, les deux visualisations reunies et sauvegardees.

Defi C : matrice de couverture MITRE ATT&CK

Le mapping MITRE ATT&CK de chacune des 4 règles maison a été recapitule puis verifie visuellement via la page native Sécurité > Règles > Couverture MITRE ATT&CK, qui confirme la couverture declaree.

Règle maison	Tactique	Technique
SSH Brute Force	Accès aux informations d'identification (TA0006)	Force brute (T1110) > Password Guessing (T1110.001)
PowerShell suspect	Exécution (TA0002)	Interpreteur de commandes et de scripts (T1059) > PowerShell (T1059.001)
Injection SQL	Accès initial (TA0001)	Exploitation de l'application publique (T1190)
Scan de ports	Découverte (TA0007)	Découverte de service réseau (T1046)

Cette répartition sur quatre tactiques distinctes (Accès initial, Exécution, Accès aux identifiants, Découverte) montre une couverture de détection etalee sur plusieurs phases d'une kill chain plutôt que concentree sur un seul type de menace -- une analyse de couverture typique d'un SOC.



Matrice de couverture MITRE ATT&CK native de Kibana : les techniques couvertes par les règles maison et importées apparaissent surlignees (Exploit Public-Facing Application, Command and Scripting Interpréter, etc.).

Point de contrôle defis

Les trois defis ont été menes a terme : une règle de faux positif documentée et volontairement classee en severite basse (Defi A), un dashboard maison fonctionnel compense l'échec d'import des visualisations Lens (Defi B), et la matrice de couverture MITRE ATT&CK confirme visuellement le mapping des 4 règles maison sur 4 tactiques distinctes (Defi C).

Conclusion

Ce TP4 a transforme la plateforme SOC-LAB-GLACIER, jusque-la capable de recherche manuelle (TP2), en un SOC produisant des alertes exploitables : RBAC au moindre privilege, import de 15 règles et de 6 dashboards, alerting fonctionnel (124 alertes observees), tuning argumente d'une règle bruyante, création de 4 règles maison couvrant 4 tactiques MITRE ATT&CK distinctes, correlation EQL démontrée et export détection-as-code effectue.

Plusieurs obstacles réels ont été rencontres et documentes plutôt que masques : une confusion d'interface entre privileges Elasticsearch et privileges Kibana lors de la création du rôle, un bug d'import constant et methodiquement isole sur les objets Saved Object de type lens (contourne par une reconstruction manuelle du dashboard du Defi B), une erreur de syntaxe KQL sur des parentheses non echappees corrigée sur une règle importee puis anticipee sur une règle maison équivalente, un oubli initial du modèle d'indexation soc-lab-events-* sur les premières règles maison (silencieusement sans alerte générée, corrige sur les quatre règles), et une erreur d'aiguillage entre deux règles de brute force au nom similaire lors du tuning.

La plateforme dispose désormais de 19 règles de détection actives, d'un dashboard d'alertes fonctionnel, d'une matrice de couverture MITRE ATT&CK vérifiée et d'un export NDJSON prêt a être verse dans un dépôt Git pour amorcer une pratique de détection-as-code. C'est sur cette base opérationnelle que s'appuiera le TP5, consacre au triage, a l'investigation, a la correlation et a la rédaction d'un rapport d'incident a partir des alertes désormais générées.